

Intrinsic Type Assignment: The Anti-Pattern That Arises When Level Membership Is Determined by Technical Implementation Rather Than Functional Role

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 12, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the instinct/reasoning separation pattern introduced in “The Instinct/Reasoning Separation Outside the Model” (Li, April 2026), the second paper in the CKS theory series following “Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems” (Li, April 2026). It does not introduce new axioms. Its contribution is to formalize one anti-pattern—Intrinsic Type Assignment—as a named failure mode of the relational and purpose-defined roles commitment (B1.17), so that downstream work can identify, detect, and remediate it with precision.

Abstract

The Coordination Knowledge Substrate (CKS) pattern, as extended in Paper 2, commits that cell, aspect, and Self are relational roles defined by functional purpose, not intrinsic types defined by technical implementation (B1.17). This commitment is operationalized through the purpose-defined level determination criteria of B2.85, the level-distinguishability tests of B2.09, and the evolution-triggered role-change mechanism of B2.87. Intrinsic Type Assignment is the anti-pattern that arises when these criteria are bypassed and level membership is instead assigned on the basis of what an entity is technically rather than what it *does* functionally. The anti-pattern appears in three recognizable sub-forms: technology-driven assignment (level follows technology choice rather than function), fixed-role entities (level is treated as immutable and never reconsidered as function evolves), and implicit type assumption (level membership is not authored in substrate but assumed from deployment context). Each sub-form produces governance misalignment—the wrong scope of commitments applied to the wrong entities—and, in the case of fixed-role entities, permanently blocks the evolution pathway B2.87 depends on. Detection runs through three checks against B2.85, B2.09, and B2.88. Remediation requires functional review of all level assignments, governance-authorized reclassification, and explicit authoring of level determination records for all entities with implicit assumptions.

1. Why this anti-pattern needs formalization

B1.17 establishes a foundational asymmetry between CKS and the modular-software traditions it builds on. Traditional modular architecture—from service-oriented architecture through microservices through

component frameworks—commits to intrinsic structural membership: a service belongs to one bounded context, a component belongs to one composite, a layer belongs to one tier. Membership is an attribute of the entity; the entity carries its structural type.

CKS's commitment in B1.17 inverts this. The same underlying CKS artifact participates as a cell in one arrangement, as part of an aspect in another, and as a component of multiple aspects simultaneously, depending on what the structure is for. The artifact's identity is the bearer; structural role-instances inhere in the bearer per the arrangement that calls on them. Level membership is not a type; it is a role that follows from function within a governed arrangement.

This asymmetry is architecturally consequential. If level membership were a type, governance commitments could be specified once at the type level and inherited statically. Because level membership is a relational role, governance scope must be determined per arrangement, per function, per purpose—and the criteria for that determination must be explicitly authored in substrate per B2.85. The criteria are functional: what task does this entity handle, what coordination purpose does this arrangement serve, what scope of governance is appropriate to that function?

Intrinsic Type Assignment is the failure mode that imports the traditional modular-architecture pattern—level as type—into a CKS deployment. When it occurs, the architectural asymmetry B1.17 establishes collapses, and governance commitments are applied by technical category rather than by functional determination. Naming the anti-pattern is what makes the collapse detectable and remediable.

2. Commitment violated

B1.17—Relational and purpose-defined roles. Cell, aspect, and Self are relational roles defined by function, not intrinsic types defined by implementation. The criteria for level membership are functional (what informational task is handled, what coordination purpose is served, what scope of integration is managed), not technical (what runtime, what LLM, what orchestration framework is used). The same underlying CKS artifact can participate at different levels in different arrangements; its level in any given arrangement is determined by its role in that arrangement, not by a type assignment made at creation.

Directly relevant decompositions: B2.84 (relational roles frame), B2.85 (purpose-defined level determination criteria), B2.87 (evolution-triggered role changes), B2.88 (relational roles verification). The level-distinguishability tests at B2.09 provide the operational boundary test whose failure the anti-pattern produces.

3. Recognizable form

Intrinsic Type Assignment appears in three distinguishable sub-forms, each representing a different mechanism by which technical type displaces functional role as the basis for level assignment.

Form 1—Technology-Driven Assignment

Level assignment is determined by technology choice rather than functional role. An entity is designated a cell because it is implemented as an LLM call wrapper; an entity is designated an aspect because it is implemented as an orchestration layer; an entity is designated a Self because it is the top-level service in

the deployment architecture. The reasoning is architectural-layer reasoning, not functional-role reasoning.

The recognizable signal is language in level determination records (or in the developer reasoning that substitutes for them) that references implementation type—LLM, orchestrator, API, microservice—rather than functional criteria per B2.85: informational task handled, coordination purpose served, integration scope managed. When a practitioner’s answer to “why is this entity at this level?” references the technology it uses rather than the function it performs, Form 1 is present.

The operational consequence is that the level-distinguishability tests of B2.09 would produce a different result if applied against functional criteria. An entity designated as an aspect because it “is an orchestration layer” may be performing a cell-scoped informational task under that orchestration infrastructure—the technology implements the right machinery, but the function belongs at a different level.

Form 2—Fixed-Role Entity

An entity’s level is treated as immutable: assigned at creation and never subject to governance review for redetermination. Even as the entity’s function accumulates and shifts through directed selection per B1.14—as operational experience shapes what the entity actually does—its level is never reconsidered.

The recognizable signal is a combination of two conditions: first, the entity has accumulated significantly different functions through directed selection, such that a fresh application of B2.85 functional criteria would yield a different level determination than the original; second, B2.87 evolution-triggered role changes have never been performed. The level determination record per B2.85, if it exists at all, reflects the entity’s function at creation time, not its current function. B2.09 level-distinguishability tests applied to the entity’s current behavior produce a result that does not match the recorded level.

Form 2 is particularly consequential because it is self-concealing. The entity continues to function, and its governance continues to operate, in ways that may appear coherent to practitioners who only observe current behavior without comparing it to the recorded level determination. The mismatch accumulates silently as the entity evolves.

Form 3—Implicit Type Assumption

An entity’s level membership is not explicitly authored in substrate at all. Level assignment exists only in deployment documentation, architectural diagrams, or developer understanding—not in substrate per B2.85/B2.08. The common pattern is a deployment-context assumption: “everything deployed in this service is a cell,” “all agents in this framework are aspects,” “the components of this pipeline are cells by default.” The assumption is categorical rather than per-entity, and it is not a substrate artifact subject to governance.

The recognizable signal is the absence of level determination records per B2.85 for individual entities. B2.08 membership records are absent; B2.88 relational roles verification finds no authored determination. What exists instead is documentation-layer or developer-knowledge-layer categorization

that is not inspectable, modifiable, or overridable through substrate governance mechanisms.

Form 3 produces a specific governance gap: because level membership is not authored in substrate, no governance action can review, verify, or update it through the substrate. When functions shift, there is no level determination record to revise. When a B2.88 verification is attempted, it finds nothing to verify. The governance affordances that should propagate per arrangement cannot propagate properly because the arrangement is not substrate-recorded.

4. Emergence conditions

Framework analogy from architectural backgrounds. Practitioners with microservices, component-framework, or service-oriented architecture backgrounds bring established pattern vocabularies for structural level. In those traditions, the question “what layer does this belong to?” is answered by examining what the component does technically—it is a service layer, an orchestration layer, a data layer. The background vocabulary maps naturally onto CKS levels as if they were technical layers, producing Form 1 at deployment time without the practitioner recognizing that a different category of question is required.

OOP type-thinking applied to structural level. Practitioners with object-oriented programming backgrounds are accustomed to structural membership as class membership: an object is an instance of its class, and class membership determines what operations apply. Applied to CKS levels, this background produces the intuition that an entity is a cell (or aspect, or Self) in the same way an object is an instance of a class—as a static type attribution rather than a relational role determination. The consequence is Form 2: once the “type” is assigned, it is treated as immutable, because class membership does not change with operational behavior.

Governance effort avoidance. Authoring per-entity level determinations per B2.85 requires explicit governance work: applying functional criteria to each entity, recording the determination in substrate, and establishing practices for revisiting determinations as functions evolve. The effort is real and bounded, but it is not automatic. Practitioners under time pressure or with low familiarity with the relational-roles commitment may substitute the lower-effort alternative of deployment-context categorical assumption, producing Form 3.

5. Operational consequences

Function-level mismatch with misapplied governance. When level is determined by technology rather than function, the governance commitments applied at that level scope may be inappropriate for the entity’s actual function. A technology-designated “aspect” that performs a cell-scoped informational task receives aspect-scope governance—aspect-level orchestration rules, aspect-level conflict handling, aspect-level composition validity requirements—when cell-scope governance is what its actual function requires. The entity’s substrate and the governance applied to it are calibrated for different purposes. Over time, this mismatch accumulates as a structural gap between what the governance provides and what the function requires.

Evolution-triggered redetermination permanently blocked. Form 2 (fixed-role entity) produces a specific failure: the B2.87 pathway by which accumulated function changes trigger level redetermination is never activated. B1.17's commitment that level membership is relational and revisable is not falsified in principle—no one has declared the commitment abandoned—but it is effectively nullified in practice because the triggering condition for revision is never examined. As directed selection shapes the entity's function over time, the gap between recorded level and functional level grows, but no mechanism surfaces the gap for governance attention.

Governance affordances applied at the wrong scope. Governance affordances that propagate per arrangement are applied at the wrong scope when level is technology-typed or categorically assumed. An entity performing cell-scope work under aspect-scope governance may have its substrate content governed by rules calibrated for coordination across multiple cells, when the appropriate governance for a single-task informational substrate is different in structure, granularity, and authority allocation. Neither form of governance is wrong in the abstract; the misapplication is in the level mismatch.

Composition validity failure. Composition requirements per A1.13, evaluated at a technology-determined level rather than a functionally-determined level, may produce inaccurate validity assessments. A composition that appears valid at the recorded level may be invalid at the functional level, or vice versa. The composition validity check runs against the wrong category, and the result cannot be trusted to identify real validity problems or real validity guarantees.

6. Detection

B2.85 purpose-defined determination criteria check. Examine the level determination records for each entity. Do those records reference functional criteria—what informational task the entity handles, what coordination purpose the arrangement serves, what integration scope is managed—or do they reference technical criteria such as runtime type, framework, or implementation architecture? Records that answer the question “what level is this?” with implementation vocabulary rather than functional vocabulary indicate Form 1. Records that are absent indicate Form 3. Records that predate significant functional evolution without subsequent revision indicate Form 2.

B2.09 level-distinguishability tests. Apply the level-distinguishability tests to each entity using its current functional behavior—what it actually does, not what its creation-time documentation says it does. Compare the result against the entity's recorded level. A mismatch between the test result and the recorded level indicates that either Form 1 (technology-driven assignment) or Form 2 (fixed-role evolution gap) is present. The test does not require access to the entity's history; it operates on current function against B2.85 criteria.

B2.88 relational roles verification. Run the B2.88 verification check against all entities in the deployment. This check requires the presence of authored level determination records per B2.85 for each entity. Entities without such records are Form 3 instances by definition. The verification also confirms that determination records reflect current function, not only creation-time function—entities that pass the existence check but fail the currency check are Form 2 candidates.

7. Remediation

Review all entity level assignments against B2.85 functional criteria. For each entity in the deployment, apply the B2.85 functional criteria directly: what informational task does this entity handle, what coordination purpose does its arrangement serve, what integration scope is relevant? This review should use current functional behavior as its input, not creation-time documentation.

Reclassify through governance-authorized redetermination where technology type does not match functional role. Where Form 1 is found—technology type and functional role diverge—reclassify through a governance-authorized redetermination process per B2.85. The redetermination is a governance act: it must be authored in substrate, attributed to an authority, and recorded with the functional criteria it applies. Reclassification is not an informal update to documentation; it is a substrate write under governance.

Author explicit level determination records per B2.85 for all entities with implicit type assumptions. Where Form 3 is found—level membership exists in documentation or developer understanding but not in substrate—author per-entity level determination records in substrate. Each record applies the B2.85 functional criteria to the entity and records the resulting determination under appropriate authority. Categorical deployment-context assumptions are not an admissible substitute.

Run B2.88 relational roles verification after remediation. Once reclassification and explicit record authoring are complete, run B2.88 verification across all entities to confirm that authored determination records are present, that they reflect current function, and that they were produced under appropriate governance authority. The verification is the operational test that the remediation is complete.

Establish governance practices that review level assignments as entity functions evolve through directed selection. The Form 2 failure mode—level treated as immutable—is prevented not by one-time remediation but by standing governance practice. Whenever directed selection substantially changes an entity's function, the question of whether the existing level determination remains accurate should be part of the governance review for that change. B2.87 evolution-triggered role changes depend on this practice being activated; they cannot activate themselves.

8. Conclusion

Intrinsic Type Assignment is the failure mode that imports traditional modular-architecture type-thinking into a CKS deployment, treating level membership as an intrinsic property of technical implementation rather than as a relational role determined by functional purpose. It appears in three sub-forms: technology-driven assignment, fixed-role entity, and implicit type assumption. Each sub-form produces governance misalignment—commitments applied at the wrong scope, evolution pathways blocked, composition validity checks yielding inaccurate results.

The anti-pattern is consequential precisely because it is natural. The background vocabularies of microservices, component frameworks, and object-oriented design all treat structural membership as a category property. Practitioners fluent in those vocabularies will default to type-thinking unless the B1.17 commitment is explicitly understood and the B2.85 functional criteria are explicitly applied. The

anti-pattern does not require bad intent or negligence; it requires only the application of a familiar vocabulary to a context that requires a different one.

Naming the anti-pattern, identifying its three sub-forms, and providing detection mechanisms through B2.85, B2.09, and B2.88 makes the departure from B1.17 operationally identifiable. Remediation through governance-authorized redetermination and explicit level determination authoring restores the relational-role structure B1.17 commits to.

Source papers

Li, W. (2026). *The Instinct/Reasoning Separation Outside the Model: Extending the Coordination Knowledge Substrate Pattern to AI Selves under Human Governance*. April 2026. ORCID: 0009-0004-8065-3235.

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Intrinsic Type Assignment: The Anti-Pattern That Arises When Level Membership Is Determined by Technical Implementation Rather Than Functional Role*. May 12, 2026. ORCID: 0009-0004-8065-3235.